yana-bhuyan / ASSIGNMENTS

Code Issues Pull requests Actions Projects Wiki Security Insig

main ASSIGNMENTS / dsa_lab_3 / experiment.md

yana-bhuyan Add files via upload db8b8e9 · now

66 lines (56 loc) · 2.05 KB

Student name : Swayam Uddeepta Bhuyan Sap id: 590021784 Batch : B-17 Course : Btech Cse Subject : DSA Experiment : 3

Experiment 3:Stack Data Structure

Objective: To demonstrate use of arrays and linked list to implement Stack operations and applications of stack.

1) Using array and functions implement Stack and its operations like insert, delete, and display.

Aim:

To implement Stack using array and perform operations such as insertion (push), deletion (pop), and display.

Algorithm:

1.Start 2.Initialize stack array and set top = -1 3.For push operation, check overflow then insert element at top 4.For pop operation, check underflow then delete element from top 5.For display, print elements from top to bottom 6.Stop

Program:

C ⊹ > main()

```c
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

// PUSH
void push(int val) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    top++;
    stack[top] = val;
}

// POP
void pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return;
    }
    top--;
}

// DISPLAY
void display() {
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }

    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);
```

```c
            printf("%d ", stack[i]);

        printf("\n");
    }

int main() {

    push(10);
    push(20);
    push(30);

    display();

    pop();
    display();

    return 0;
}
```

Compilation and Execution:

```
HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_3
$ gcc -o 1 1.c

HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_3
$ ./1
30 20 10
20 10
```

Observation:

Stack follows LIFO principle.

Insertion and deletion occur only at the top.

Overflow occurs when stack is full.

Underflow occurs when stack is empty.

Array stack has fixed size.

Result:

main ASSIGNMENTS / dsa_lab_3 / experiment.md Top

Preview Code Blame Raw

2)Reverse a string using stack.

Aim:

To reverse a string using stack data structure.

Algorithm:

1.Start 2.Read the input string 3.Push each character of the string into stack 4.Pop characters from stack one by one 5.Form reversed string using popped characters 6.Display reversed string 7.Stop

Program:

C 2 > main()

```c
#include <stdio.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

// PUSH
void push(char c) {
    stack[++top] = c;
}

// POP
char pop() {
    return stack[top--];
}

int main() {
    char str[100];

    printf("Enter string: ");
    scanf("%s", str);

    int n = strlen(str);

    // Push all characters
    for (int i = 0; i < n; i++)
        push(str[i]);

    // Pop to reverse
    for (int i = 0; i < n; i++)
        str[i] = pop();
```

```
    // Pop to reverse
    for (int i = 0; i < n; i++)
        str[i] = pop();

    printf("Reversed string: %s\n", str);

    return 0;
}
```

Compilation and Execution:

```
HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_3
$ gcc -o 2 2.c

HP@DESKTOP-7K7BG1T MINGW64 ~/OneDrive - UPES/Desktop/dsa_c/dsa_lab_3
$ ./2
Enter string: ers
Reversed string: sre
```

Observation:

Stack follows LIFO which makes reversal easy.

Each character is pushed then popped in reverse order.

Stack is useful in string manipulation.

Demonstrates application of stack.

Result:

The given string was successfully reversed using stack.